Fintech Audit: Reconciling a Disputed Bank Debit

- Batch 1 · 23I501 Design and Analysis of Algorithms · PSG College of Technology
- A customer disputes a debit of ₹1,10,000; no single pending transaction matches
- Pending transactions (₹ thousand): 15, 22, 35, 40, 48, 53, 61
- Task: report EVERY combination whose total is exactly 110 (real batches: 30–40 transactions)
- Part 1 (Madhumitha N, 24I328): model, technique, algorithm, trace, proof, complexity
- Part 2 (Nivedhini M, 24I335): rejected techniques, experiments, ±₹50 change, reference

Presented by: Madhumitha N (24I328) · Nivedhini M (24I335)

Problem Modelling

Input	**n positive integer amounts (in rupees), target T = 1,10,000, tolerance τ (0 now, ₹50 later)**
Output	ALL non-empty subsets of transactions with $T - \tau \leq sum \leq T + \tau$
Objective	Completeness: none missed, none invalid, none repeated
Constraints	Amounts > 0 · each transaction used at most once · equal amounts are different items · integer rupees, no floats
Class	Enumeration (listing). Decision version is NP-complete; #solutions can be exponential
Classical problem	SUBSET SUM, all-solutions listing form

- Why “every combination” matters: yes/no tables and one-path greedy cannot produce a list — we need an exhaustive generator with SAFE cut-offs

Data Structures and Technique Choice

Structure	Cost	Space	n=40, T≈10^9 paise
Sorted array + suffix sums (CHOSEN)	O(1) per step	O(n)	independent of T
DP table / bitset reach[i][s]	O(n·T) build	O(n·T)	≈4.1×10^{10} cells ≈ 41 GB (5.1 GB as bits)
Meet-in-the-middle sorted lists	O(n·2^(n/2))	O(2^(n/2))	2^{20} ≈ 10^6 sums per half: feasible
Bitmask over all subsets	O(n) per mask	O(1)	2^{40} masks: infeasible

- TECHNIQUE: Backtracking (depth-first, build subsets in increasing order)
- Enabling property: with positive amounts, a partial sum BOUNDS every completion
- Rule A overshoot: c + v_j > hi ⇒ every later item overshoots (sorted)
- Rule B unreachable: c + suffix[j] < lo ⇒ nothing from j onward can reach lo

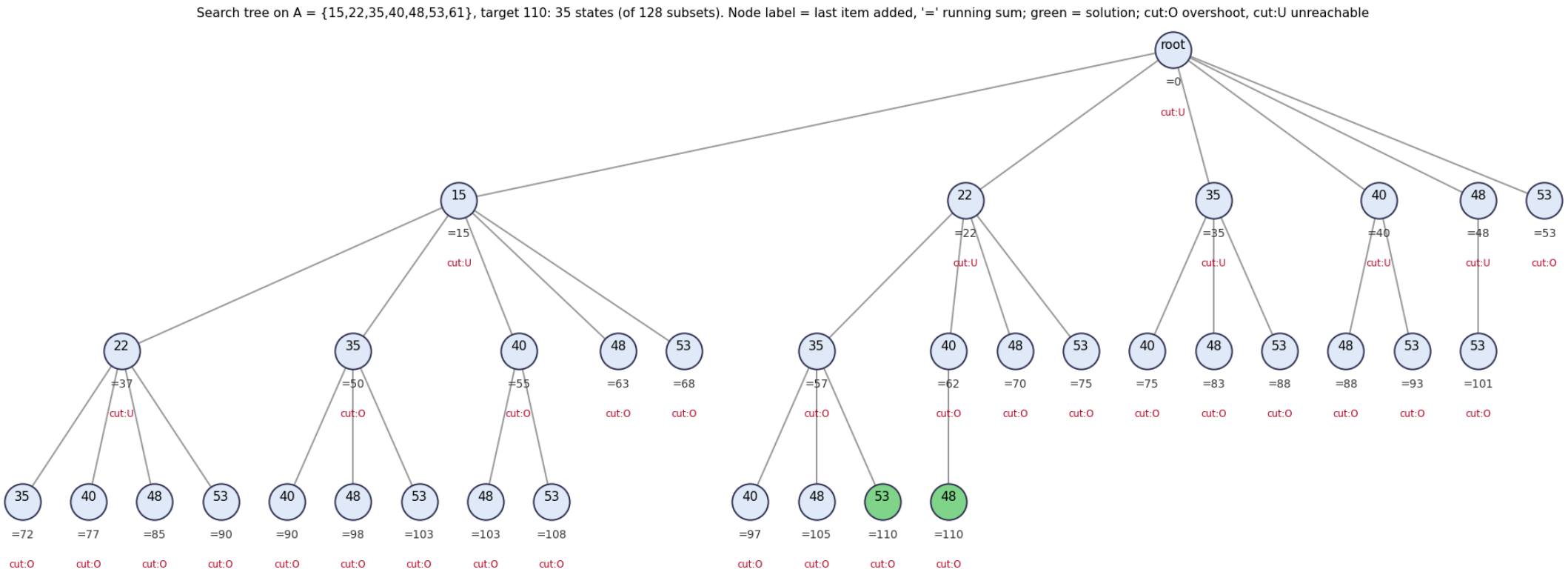
The Algorithm

- State = one recursive call = one candidate subset (start, c): c = its sum, start = first position that may still be added
- Children: C ∪ {j}, for j = start … n−1 in increasing order → every subset is built in exactly one way
- Report C if it is non-empty and lo ≤ c ≤ hi (lo = T − τ, hi = T + τ)

```
SEARCH(start, c):
    states += 1
    if chosen ≠ ∅ and lo ≤ c ≤ hi:  record solution
    for j = start .. n-1:
        if c + suffix[j] < lo:  break      # unreachable
        if c + v[j]    > hi:    break      # overshoot
        push j;  SEARCH(j+1, c + v[j]);  pop
```

- Items sorted ascending, ties by original index (duplicates stay separate items)
- Counts: states, overshoot cut-offs, unreachable cut-offs — measured separately; printing is outside the algorithm

Trace on the Given Instance



- ANSWER: {22, 35, 53} and {22, 40, 48} (both sum to 110)
- States visited = candidate subsets tested: 35 of $2^7 = 128$ (1 / 6 / 15 / 13 by size 0–3)
- Cut-offs: 28 overshoot + 7 unreachable, skipping 56 child branches · 93 subsets never generated
- Example: {15,22,35} = 72, next 40 → 112 > 110: overshoot. {35}: 35 + 61 = 96 < 110: unreachable

Proof of Correctness

- Lemma A: if $c + v_j > hi$, every non-empty $D \subseteq \{j..n-1\}$ gives sum $\geq c + v_j > hi$ (sorted, positive)
- Lemma B: if $c + suffix[j] < lo$, every $D \subseteq \{j..n-1\}$ gives sum $\leq c + suffix[j] < lo$ (positive)
- Theorem (induction on $n - start$): SEARCH(start, c) reports exactly the valid sets $C \cup D$, $D \subseteq \{start..n-1\}$, each once
- Base: $start = n \rightarrow$ only C itself, tested at entry. Step: split by smallest added j; no cut $\Rightarrow$ induction hypothesis; cut $\Rightarrow$ Lemma A/B (also for all $j' \geq j$)
- Complete: apply the Theorem at the root ($C = \emptyset$). Sound: report only after the window test; positions strictly increase $\Rightarrow$ no item twice
- No duplicates: subset $\leftrightarrow$ one increasing sequence; parent = remove largest position
- Same proof for any $\tau \geq 0$: only lo and hi change

Complexity and a Hard Input

- Time = Θ(states) + O(n log n) sort + O(output size); states ≤ min(2^n, Σ_{k≤K} C(n,k)), K = most items that can fit
- Best Θ(n log n) · Worst Θ(2^n) · Space O(n) + output. Given instance: K = 3 ⇒ ≤ 64 states, measured 35
- Practical: target small vs batch total ⇒ small K ⇒ few states (n = 40 random batches: median 5.6×10^5, max 8.2×10^6 vs $2^{40} \approx 1.1\times10^{12}$)
- Output can be exponential (equal amounts: C(n, n/2) answers) ⇒ no algorithm beats the output size
- Hard family (no solution): $a_i = M + i$, $M = 10n^2$, $T = (\lfloor n/2 \rfloor + 1)M - 1$

n	states	C(n,n/2)	2^n	states/2^n
14	3,432	3,432	16,384	0.209
18	48,620	48,620	262,144	0.185
22	705,432	705,432	4,194,304	0.168

- Exactly C(n, ⌊n/2⌋) states (derived, verified for n = 4..22): pruning saves only ≈ √(πn/2). True worst case 2^n: everything is a solution

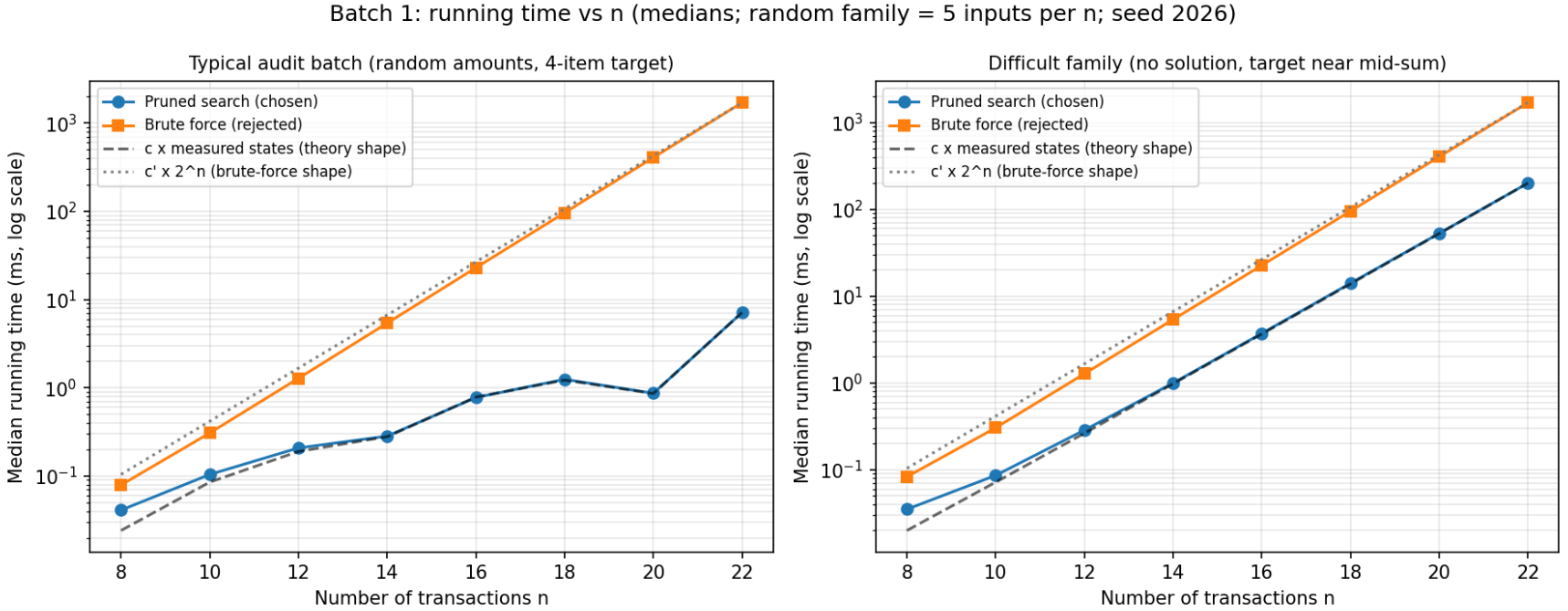
Rejected: Greedy and Dynamic Programming

- GREEDY (largest that still fits): 61 → remaining 49 → 48 → remaining 1 → FAILS, reports nothing
- Yet {22,35,53} and {22,40,48} exist; neither contains 61 ⇒ greedy-choice property false at step 1 (49 is not a subset sum of the rest)
- A single path can output at most one subset; we need all
- DYNAMIC PROGRAMMING: reach[i][s] answers “is s reachable?”; listing needs all rows + a walk-back, O(n) per solution — DP CAN enumerate
- But cost depends on the VALUE of T (pseudo-polynomial): n = 40, T ≈ 10^9 paise ⇒ 4.1×10^{10} cells = 41 GB (5.1 GB as bits)
- Given instance: table 888 cells vs 35 search states. Honest note: $2^{40} \approx 1.1\times10^{12} > n\cdot T$, so DP has the better worst-case count — it loses on memory and on realistic batches (~5×10^5 states)

Rejected: Divide and Conquer, Branch and Bound

- DIVIDE AND CONQUER = meet-in-the-middle (Horowitz–Sahni): two halves of 20 → $2^{20} \approx 10^6$ sums each, sort, match
- Worst case O(n·2^(n/2)): far better than 2^{40} — it is a strong alternative (we cross-checked it against our solver)
- Not primary because: the COMBINE step (matching every left sum with right sums) is the cost and does not recurse; it always uses 2^(n/2) time AND memory; audit targets are small so pruning keeps the DFS tiny; DFS memory is O(n) and traceable
- BRANCH AND BOUND: needs an objective + incumbent; here the only bounds are feasibility bounds (the same two rules)
- Best-first order needs a priority queue and frontier memory; states are the same, so extra cost for no gain
- Choice: plain depth-first backtracking = simplest, O(n) memory — not because the others cannot solve it

Experiments: Pruned Search vs Brute Force



- n = 8…22 (both), 24…40 (chosen only) · same inputs · seed 2026 · median of repeats · printing outside timing · solution sets compared first: identical in every case
- n = 22: brute force 1,710 ms vs search 7.1 ms on random batches (≈239×); on the hard family 1,700 ms vs 201 ms (≈8.5×)
- Brute force grows ≈ 4.2× per +2 in n ($n \cdot 2^n$). Hard family grows ≈ 3.8× per +2 = C(n,n/2); dashed line = c × states lies on the data ⇒ time ∝ states
- Random family is irregular (n = 20 faster than n = 18): states depend on target vs total, not on n alone

Changed Requirement: ±₹50 Tolerance

- Units: amounts in thousands → integer rupees. T = 110,000 · τ = 50 (= 0.05 thousand), exact via Fraction, no floats
- New condition: 109,950 ≤ sum ≤ 110,050
- Change: only lo = T − τ and hi = T + τ; same two rules; proof unchanged; solutions may now extend other solutions
- Given instance: all sums are multiples of ₹1,000 ⇒ still the same 2 solutions, still 35 states

τ (₹)	**0**	**50**	**500**	**5,000**	**50,000**
solutions	3	61	605	5,920	64,963
states	24,580	24,609	24,901	27,990	72,530

- Random n = 22 batch. Hypothetical amounts [15020, 22000, 35000, 39990, 48000, 53030, 61000]: τ = 0 → 0 solutions; τ = 50 → 2 (110,030 and 109,990)
- Complexity: still O(n log n + states + output), worst case $\Theta(2^n)$; output grows ≈ in proportion to the window width

Real-World Reference

- Horowitz & Sahni, “Computing Partitions with Applications to the Knapsack Problem”, J. ACM 21(2):277–292, 1974 — doi:10.1145/321812.321823
- Studies finding ALL combinations of numbers that sum to M; compares the usual algorithms and presents a square-root-factor improvement over dynamic programming (two-list / meet-in-the-middle idea)
- Ours: simple DFS with two bounds, O(n) memory, $\Theta(2^n)$ worst case — theirs: 2^(n/2) techniques with more elaborate heuristics (we do not reproduce them)
- Technique origin: Golomb & Baumert, “Backtrack Programming”, J. ACM 12(4):516–524, 1965 — doi:10.1145/321296.321300
- Python itertools.combinations: generic enumerator, used only in our brute-force baseline

Conclusion

- Problem: Subset Sum, listing form → Backtracking with two proven-safe cut-offs
- Answer: {22,35,53} and {22,40,48}; 35 of 128 subsets examined
- Rejected with evidence: Greedy (wrong), DP (value-dependent memory), D&C (strong but heavier combine), B&B (no objective)
- Measured: ≈239× faster than brute force on typical batches at n = 22; only ≈8.5× on the hard family — exponential worst case remains
- ±₹50: same technique, two constants change; exact integer arithmetic
- Limitations: worst case $\Theta(2^n)$; for adversarial n = 40, meet-in-the-middle is a better guarantee